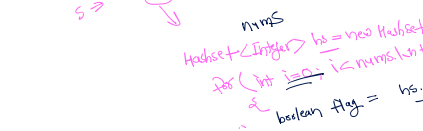


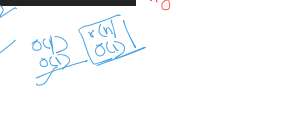
Handset <String> mail = new HandsetC();
mail.add("ak@es.com");
~~mail.add("ak@es.com");~~



↓ } stem m.



```
class Solution {
    public boolean containsDuplicate(int[] nums) {
        HashSet<Integer> set = new HashSet<>();
        for(int i = 0; i < nums.length; i++) {
            if (set.contains(nums[i])) {
                return true;
            }
            set.add(nums[i]);
        }
        return false;
    }
}
```



$O(1)$
 $O(n)$
 $O(n)$
 space $\Rightarrow O(n)$

```

HashMap<int, int> map = new HashMap<>();
for (int i = 0; i < arr.length; i++) {
    int expect = target - arr[i];
    if (map.containsKey(expect)) {
        return new int[] { map.get(expect), i };
    }
    map.put(arr[i], i);
}
    
```

$7 = \text{target}$

2	3	11	6
0	1	2	3

$7 - 5 = 2 \times$
 $7 - 3 = 4 \times$
 $7 - 2 = 5$

$5 \rightarrow 0$
 $3 \rightarrow 1$

$\text{target} = 17$
 $17 - 6 = 11$

rememb arr
 used hashu $\Rightarrow$

$2 \rightarrow 0$
 $3 \rightarrow 1$
 $11 \rightarrow 2$

2	3	11	7
0	1	2	3

$2 + 3 = 5 \times$
 $2 + 11 = 13 \times$
 $2 + 7 = 9 \times$
 $3 + 11 = 14 \times$
 $3 + 7 = 10 \times$
 $11 + 7 = 18$

$\text{target} = 17$

Basic force
 $\text{int[] ans} = \text{new int[2]};$
 for (int i = 0; i < arr.length; i++) {
 for (int j = i + 1; j < arr.length; j++) {
 if (arr[i] + arr[j] == target) {
 ans[0] = i;
 ans[1] = j;
 }
 }
 }
 return ans;

worst $\Rightarrow n^2$

.

for (i = length; i <= 0; i--)

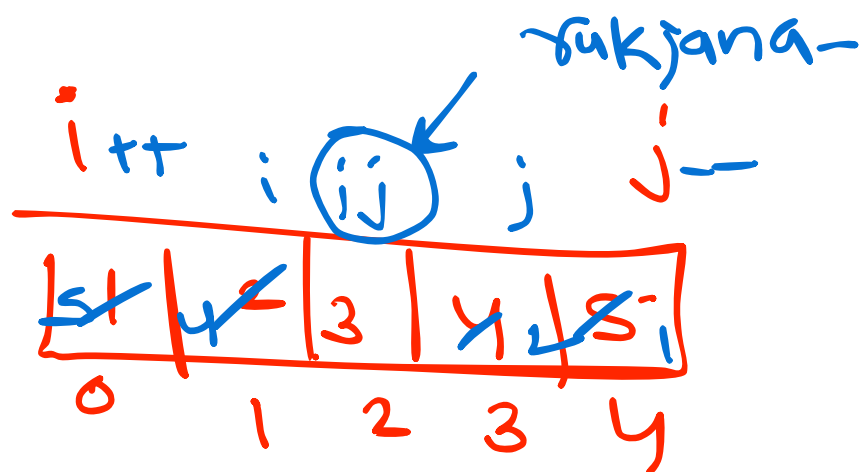
1	2	3	4
0	1	2	3

$\text{newArray}(i - \text{length} + 1) = 4$

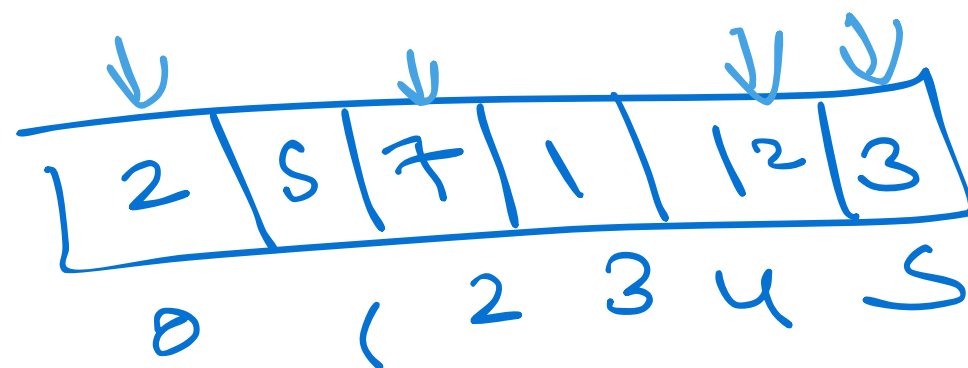
$0 \rightarrow 3$
 $1 \rightarrow 2$
 $2 \rightarrow 1$
 $3 \rightarrow 0$

newArray

two pointers
3



Swap
arr[i] arr[j]



max = 12

second max = 7

n + n = 2n
sm = 7

if (arr[0] != max)

second max = arr[0]

for (int i = 0; i < n; i++)

if (arr[i] > second max && arr[i] != max)

second max = arr[i]

12 != 12

12 != 12

return

